Flight Delay Prediction

36-673 Final Project

Ellie Huang (elliehua) and Nikita Jayaprakash (njayapra)

2026-04-22

Data Prep

Load Libraries and Data

```
# Load data
flights2023 <- read_csv(here("data", "flights2023.csv")) |>
 clean_names()

flights2024 <- read_csv(here("data", "flights2024.csv")) |>
 clean_names()

test <- read_csv(here("data", "flights2025.csv")) |>
 clean_names()

test_visible <- read_csv(here("data", "flights2025_visible.csv")) |>
 clean_names() |>
  mutate(data_year = 2025)

test_guess <- read_csv(here("data", "flights2025_guess.csv")) |>
 clean_names() |>
  mutate(data_year = 2025)

# Build training set
train <- bind_rows(
 flights2023 |> mutate(data_year = 2023),
 flights2024 |> mutate(data_year = 2024)
)
```

Clean and Filter Data

```
# Apply the same cleaning steps to each data frame so train and test sets are consistent
clean_flights <- function(df) {
 df |>
 clean_names() |>
 mutate(
 fl_date = as.Date(fl_date),
```

```r
 dep_del15 = as.numeric(dep_del15),
 op_unique_carrier = as.factor(op_unique_carrier),
 origin = as.factor(origin),
 dest = as.factor(dest),
 sched_dep_hour = floor(crs_dep_time / 100),
 sched_arr_hour = floor(crs_arr_time / 100),
 month = as.factor(month),
 day_of_week = as.factor(day_of_week)
 )
}

train <- clean_flights(train)
test <- clean_flights(test)
test_visible <- clean_flights(test_visible)
test_guess <- clean_flights(test_guess)
```

```r
# Restrict to departures from PIT
train <- train |>
 filter(origin == "PIT")

test_visible <- test_visible |>
 filter(origin == "PIT")

test_guess <- test_guess |>
 filter(origin == "PIT")

# Quick check of the target distribution
table(train$dep_del15, useNA = "ifany")
```

```
##
##     0     1  <NA>
## 59534 10972   950
```

```r
# Inbound flight features
# For each PIT departure, we try to match the same aircraft (tail + date) arriving at PIT
# earlier that day. Match rates: ~55% for training, ~15% for guess flights.
# For guess, only arrivals in flights2025_visible.csv have known arr_delay -
# guess-period arrivals are blanked out. The 85% with no match are overnighted
# or first-flight aircraft, which is itself informative (lower delay risk).
arrivals_train <- bind_rows(
  clean_flights(read_csv(here("data", "flights2023.csv"), show_col_types = FALSE)) |>
    filter(dest == "PIT") |> mutate(data_year = 2023),
  clean_flights(read_csv(here("data", "flights2024.csv"), show_col_types = FALSE)) |>
    filter(dest == "PIT") |> mutate(data_year = 2024)
)

# Only visible arrivals have non-NA arr_delay bc guess-period arrivals are blanked out
arrivals_test_visible <- clean_flights(
  read_csv(here("data", "flights2025_visible.csv"), show_col_types = FALSE)
) |> filter(dest == "PIT") |> mutate(data_year = 2025)

# For each (date, tail_num), keep the latest arriving flight as the inbound leg
make_inbound_lookup <- function(arrivals_df) {
```

```r
  arrivals_df |>
    filter(!is.na(tail_num), tail_num != "", !is.na(arr_delay)) |>
    group_by(fl_date, tail_num) |>
    slice_max(crs_arr_time, n = 1, with_ties = FALSE) |>
    ungroup() |>
    select(fl_date, tail_num,
           inbound_arr_delay = arr_delay,  # minutes late on arrival
           inbound_arr_del15 = arr_del15,  # binary: arrived >= 15 min late
           inbound_crs_arr_time = crs_arr_time)  # scheduled arrival time
}

inbound_train <- make_inbound_lookup(arrivals_train)
inbound_visible <- make_inbound_lookup(arrivals_test_visible)
inbound_guess <- inbound_visible  # guess set uses visible-only lookup

join_inbound <- function(dep_df, inbound_df) {
  dep_df |>
    left_join(inbound_df, by = c("fl_date", "tail_num")) |>
    mutate(
      # min between scheduled inbound arrival and scheduled departure
      turnaround_mins = sched_dep_hour * 60 + (crs_dep_time %% 100) -
                        (inbound_crs_arr_time %/% 100 * 60 + inbound_crs_arr_time %% 100),
      # indicator (< 45 min scheduled buffer is risky)
      tight_turnaround = if_else(!is.na(turnaround_mins) & turnaround_mins < 45, 1L, 0L),
      # indicator (whether we found a matching inbound leg at all)
      # has_inbound = 0 could mean overnighted OR inbound after cutoff in guess set
      # so we keep it as context but don't include it in model_vars
      has_inbound = if_else(!is.na(inbound_arr_delay), 1L, 0L),
      # fill unmatched rows with 0 (no evidence of inbound delay)
      inbound_arr_delay = coalesce(as.numeric(inbound_arr_delay), 0),
      inbound_arr_del15 = coalesce(as.numeric(inbound_arr_del15), 0),
      turnaround_mins = coalesce(turnaround_mins, 0)
    )
}

train <- join_inbound(train, inbound_train)
test_visible <- join_inbound(test_visible, inbound_visible)
test_guess <- join_inbound(test_guess, inbound_guess)
```

We focus on Pittsburgh departures because that is the airport of interest in the prediction task. We also use `sched_dep_hour` rather than actual departure time, since actual departure time isn't available for the flights we need to predict.

Exploratory Data Analysis

The goal of the EDA is to identify patterns that are both predictive and available before departure. In particular, we want to see whether delay risk depends in a stable way on scheduling, carrier, route, seasonality, and same-day operating conditions.

Missingness

```r
# Missingness in the main fields
colSums(is.na(train[, c("dep_del15", "dep_time", "crs_dep_time")]))
```

```
##    dep_del15     dep_time crs_dep_time
##          950          950            0
```

```r
# Variables with nonzero missingness
train |>
 summarise(across(everything(), ~mean(is.na(.)))) |>
 pivot_longer(everything(), names_to = "variable", values_to = "pct_missing") |>
 filter(pct_missing > 0) |>
 arrange(desc(pct_missing)) |>
 print(n = 30)
```

```
## # A tibble: 28 x 2
##    variable             pct_missing
##    <chr>                      <dbl>
##  1 first_dep_time           0.995
##  2 total_add_gtime          0.995
##  3 longest_add_gtime        0.995
##  4 cancellation_code        0.986
##  5 carrier_delay            0.837
##  6 weather_delay            0.837
##  7 nas_delay                0.837
##  8 security_delay           0.837
##  9 late_aircraft_delay      0.837
## 10 fl_date                  0.609
## 11 inbound_crs_arr_time     0.161
## 12 arr_delay                0.0154
## 13 arr_delay_new            0.0154
## 14 arr_del15                0.0154
## 15 arr_delay_group          0.0154
## 16 actual_elapsed_time      0.0154
## 17 air_time                 0.0154
## 18 wheels_on                0.0138
## 19 taxi_in                  0.0138
## 20 arr_time                 0.0138
## 21 taxi_out                 0.0137
## 22 wheels_off               0.0137
## 23 dep_time                 0.0133
## 24 dep_delay                0.0133
## 25 dep_delay_new            0.0133
## 26 dep_del15                0.0133
## 27 dep_delay_group          0.0133
## 28 tail_num                 0.00158
```

```r
# Missingness among key pre-departure predictors
key_predictors <- c(
 "dep_del15", "crs_dep_time", "sched_dep_hour", "op_unique_carrier",
 "dest", "distance", "month", "day_of_week", "flights"
```

```r
)

key_predictors <- intersect(key_predictors, names(train))

train |>
 summarise(across(all_of(key_predictors), ~mean(is.na(.)))) |>
 pivot_longer(everything(), names_to = "variable", values_to = "pct_missing") |>
 mutate(pct_missing = percent(pct_missing, accuracy = 0.1))
```

```
## # A tibble: 9 x 2
##   variable          pct_missing
##   <chr>             <chr>
## 1 dep_del15         1.3%
## 2 crs_dep_time      0.0%
## 3 sched_dep_hour    0.0%
## 4 op_unique_carrier 0.0%
## 5 dest              0.0%
## 6 distance          0.0%
## 7 month             0.0%
## 8 day_of_week       0.0%
## 9 flights           0.0%
```

```r
# Check whether missing targets line up with likely cancellations
train |>
 mutate(cancelled_proxy = is.na(dep_time) & is.na(dep_del15)) |>
 count(cancelled_proxy, missing_target = is.na(dep_del15))
```

```
## # A tibble: 2 x 3
##   cancelled_proxy missing_target     n
##   <lgl>           <lgl>          <int>
## 1 FALSE           FALSE          70506
## 2 TRUE            TRUE             950
```

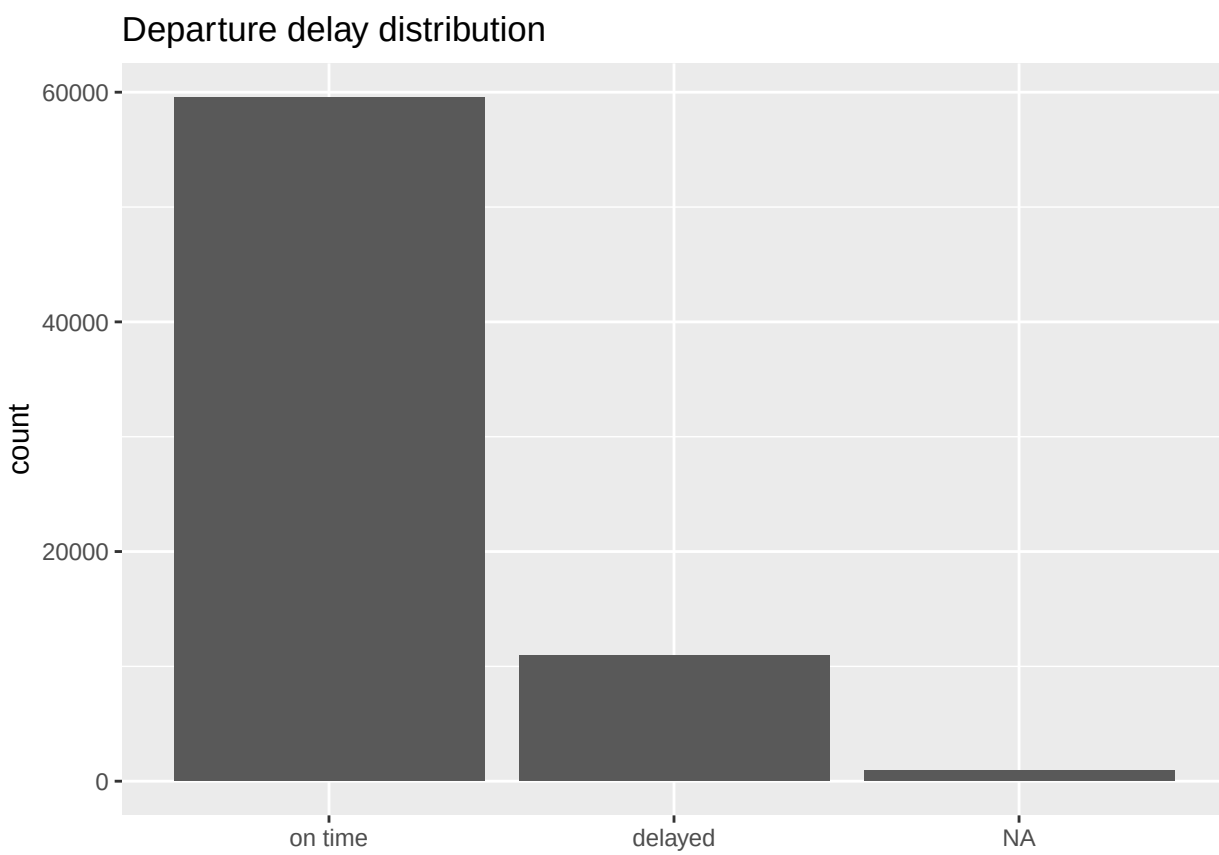
Missing target values align exactly with likely cancellations, so this is not unexplained missingness. We therefore treat these rows as cancelled flights and exclude them from the delay prediction task.

Departure Delay Distribution

```r
# Target distribution
train |>
 count(dep_del15)
```

```
## # A tibble: 3 x 2
##   dep_del15     n
##       <dbl> <int>
## 1         0 59534
## 2         1 10972
## 3        NA   950
```

```r
train |>
 ggplot(aes(x = factor(dep_del15, labels = c("on time", "delayed")))) +
 geom_bar() +
 labs(
 title = "Departure delay distribution",
 x = NULL,
 y = "count"
 )
```

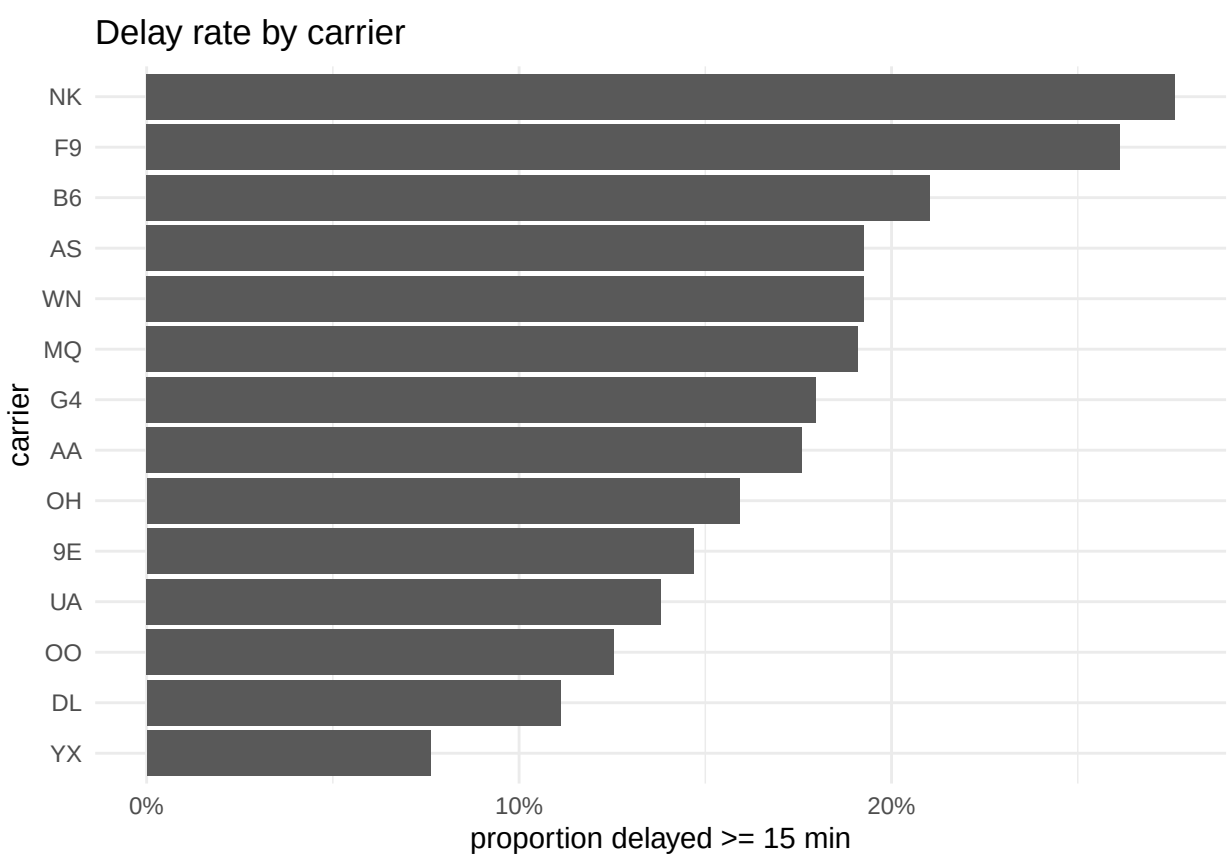


The classes are roughly 80/20 on-time versus delayed. This imbalance is worth keeping in mind, but it is not too concerning since our main evaluation metric is AUC rather than raw accuracy.

Delay Rate by Carrier

```r
carrier_delay_tbl <- train |>
 filter(!is.na(dep_del15)) |>
 group_by(op_unique_carrier) |>
 summarise(
 delay_rate = mean(dep_del15),
 n = n(),
 .groups = "drop"
 ) |>
 arrange(desc(delay_rate))
```

```r
carrier_delay_tbl |>
 ggplot(aes(x = reorder(op_unique_carrier, delay_rate), y = delay_rate)) +
 geom_col() +
 coord_flip() +
 scale_y_continuous(labels = percent_format()) +
 labs(
 title = "Delay rate by carrier",
 x = "carrier",
 y = "proportion delayed >= 15 min"
 ) +
 theme_minimal()
```

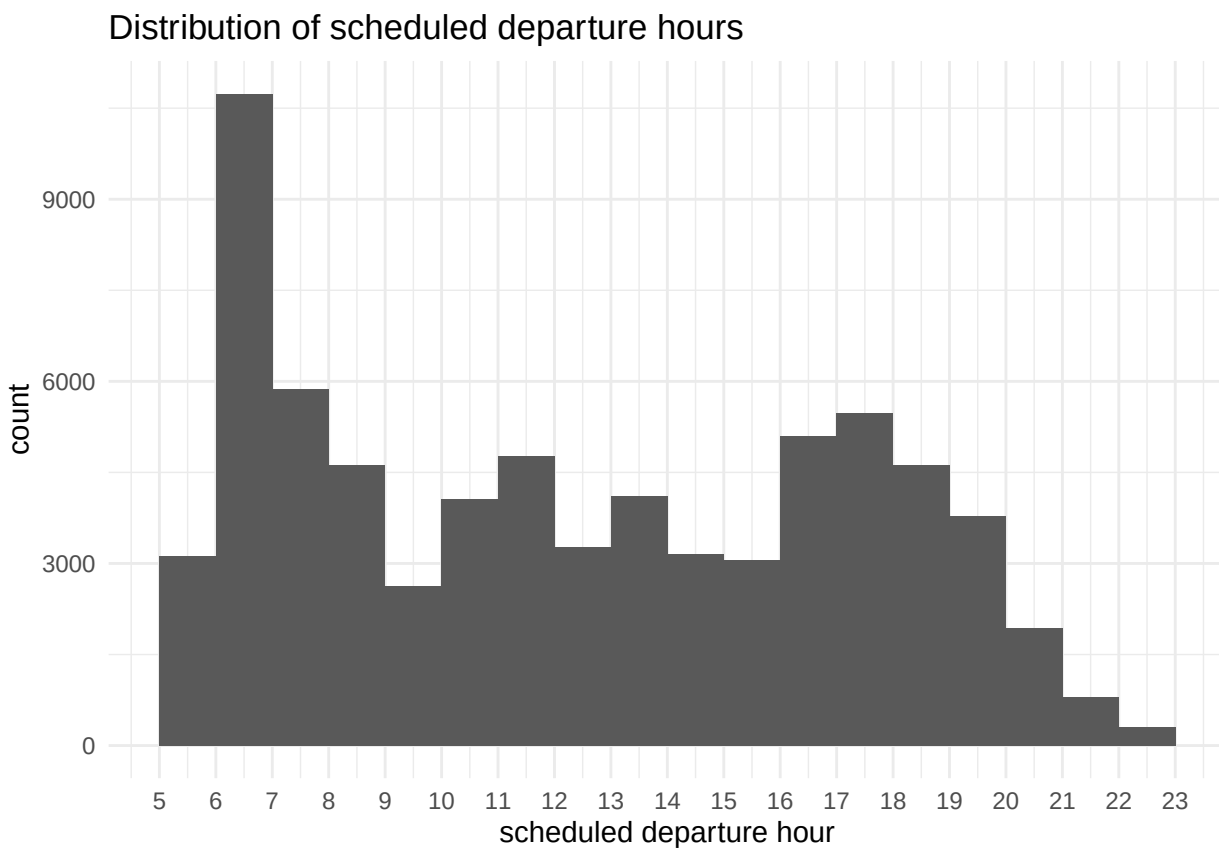


Delay rates vary meaningfully across carriers, so carrier should be a useful predictor. At the same time, some carriers have much lower flight counts at PIT than others, so raw group means will be noisy. That motivates using smoothed historical delay rates later in feature engineering instead of unsmoothed averages.

Delay Rate by Scheduled Departure Hour

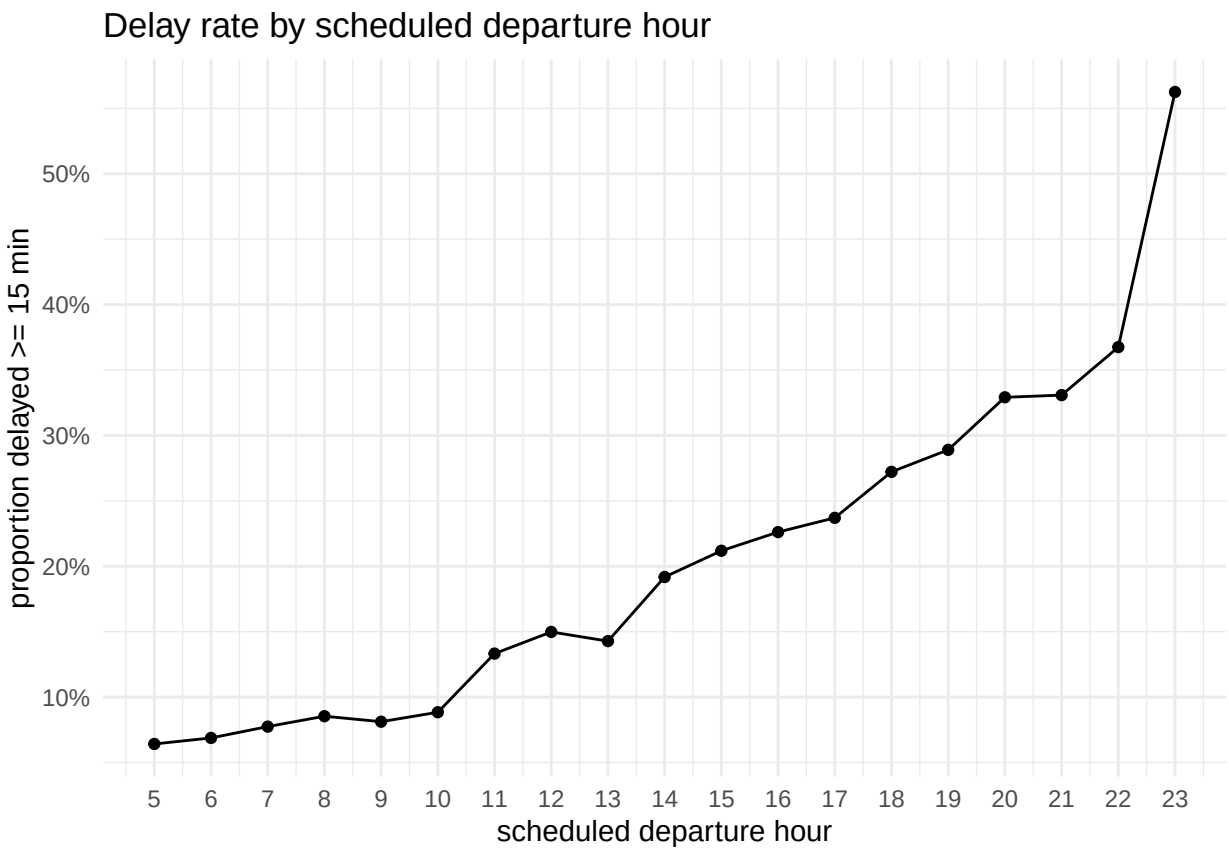
```r
# Distribution of scheduled departure times
train |>
 ggplot(aes(x = sched_dep_hour)) +
 geom_histogram(binwidth = 1, boundary = 0, closed = "left") +
 scale_x_continuous(breaks = 0:23) +
 labs(
 title = "Distribution of scheduled departure hours",
```

```
 x = "scheduled departure hour",
 y = "count"
 ) +
 theme_minimal()
```




```
hour_delay_tbl <- train |>
 filter(!is.na(dep_del15), !is.na(sched_dep_hour)) |>
 group_by(sched_dep_hour) |>
 summarise(
 delay_rate = mean(dep_del15),
 n = n(),
 .groups = "drop"
 )

hour_delay_tbl |>
 ggplot(aes(x = sched_dep_hour, y = delay_rate)) +
 geom_line() +
 geom_point() +
 scale_x_continuous(breaks = 0:23) +
 scale_y_continuous(labels = percent_format()) +
 labs(
 title = "Delay rate by scheduled departure hour",
 x = "scheduled departure hour",
 y = "proportion delayed >= 15 min"
 ) +
 theme_minimal()
```



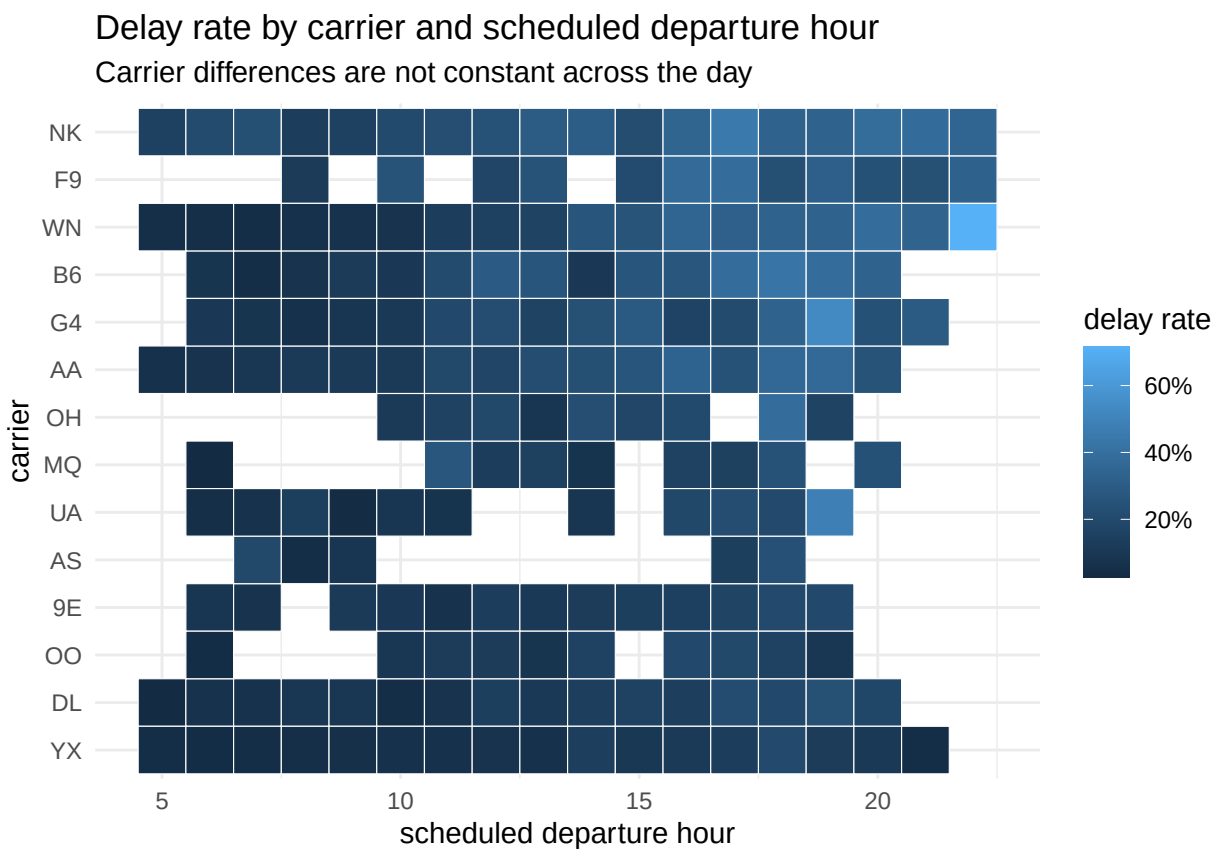
Delay rates rise steadily over the course of the day and are highest in the evening. This fits the idea of cascading delays: early flights start the day with fewer disruptions from previous legs, while later flights are more exposed to effects from earlier problems. This makes scheduled departure hour one of the strongest single predictors in the dataset.

Carrier-by-Hour Interaction

```
carrier_hour_tbl <- train |>
 filter(!is.na(dep_del15), !is.na(sched_dep_hour)) |>
 group_by(op_unique_carrier, sched_dep_hour) |>
 summarise(
 delay_rate = mean(dep_del15),
 n = n(),
 .groups = "drop"
 ) |>
 filter(n >= 20)

carrier_hour_tbl |>
 ggplot(
 aes(
 x = sched_dep_hour,
 y = fct_reorder(op_unique_carrier, delay_rate, .fun = mean),
 fill = delay_rate
 )
 ) +
```

```r
geom_tile(color = "white") +
scale_fill_gradient(labels = percent_format(accuracy = 1)) +
labs(
title = "Delay rate by carrier and scheduled departure hour",
subtitle = "Carrier differences are not constant across the day",
x = "scheduled departure hour",
y = "carrier",
fill = "delay rate"
) +
theme_minimal()
```

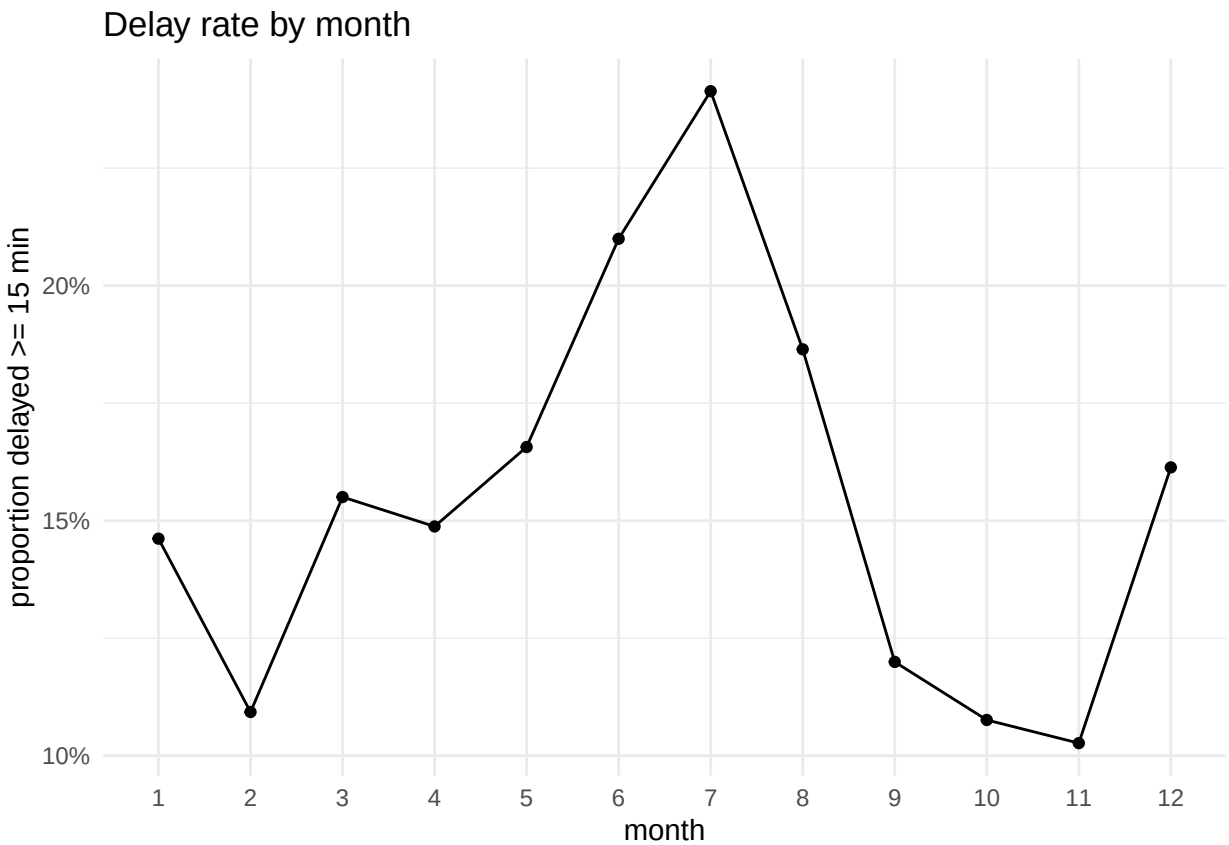


The carrier effect is not the same at every hour. Some carriers remain fairly stable across the day, while others become much more delay-prone in the evening. This suggests that a model with only main effects may miss meaningful structure, so either an explicit carrier-by-hour feature or a nonlinear model should help.

Seasonality and Calendar Effects

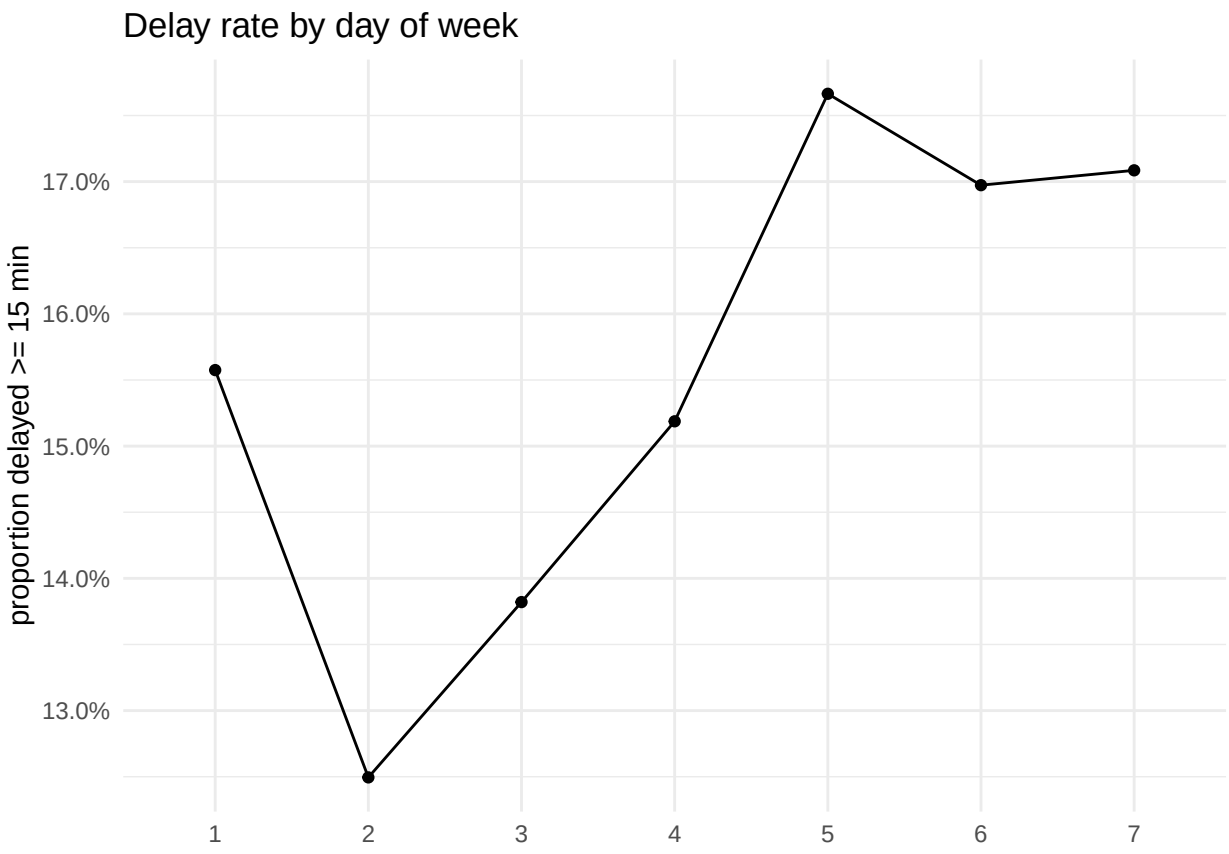
```r
# Delay rate by month
train |>
 filter(!is.na(dep_del15)) |>
 group_by(month) |>
 summarise(
 delay_rate = mean(dep_del15),
 .groups = "drop"
```

```
) |>
ggplot(aes(x = month, y = delay_rate, group = 1)) +
geom_line() +
geom_point() +
scale_y_continuous(labels = percent_format()) +
labs(
title = "Delay rate by month",
x = "month",
y = "proportion delayed >= 15 min"
) +
theme_minimal()
```

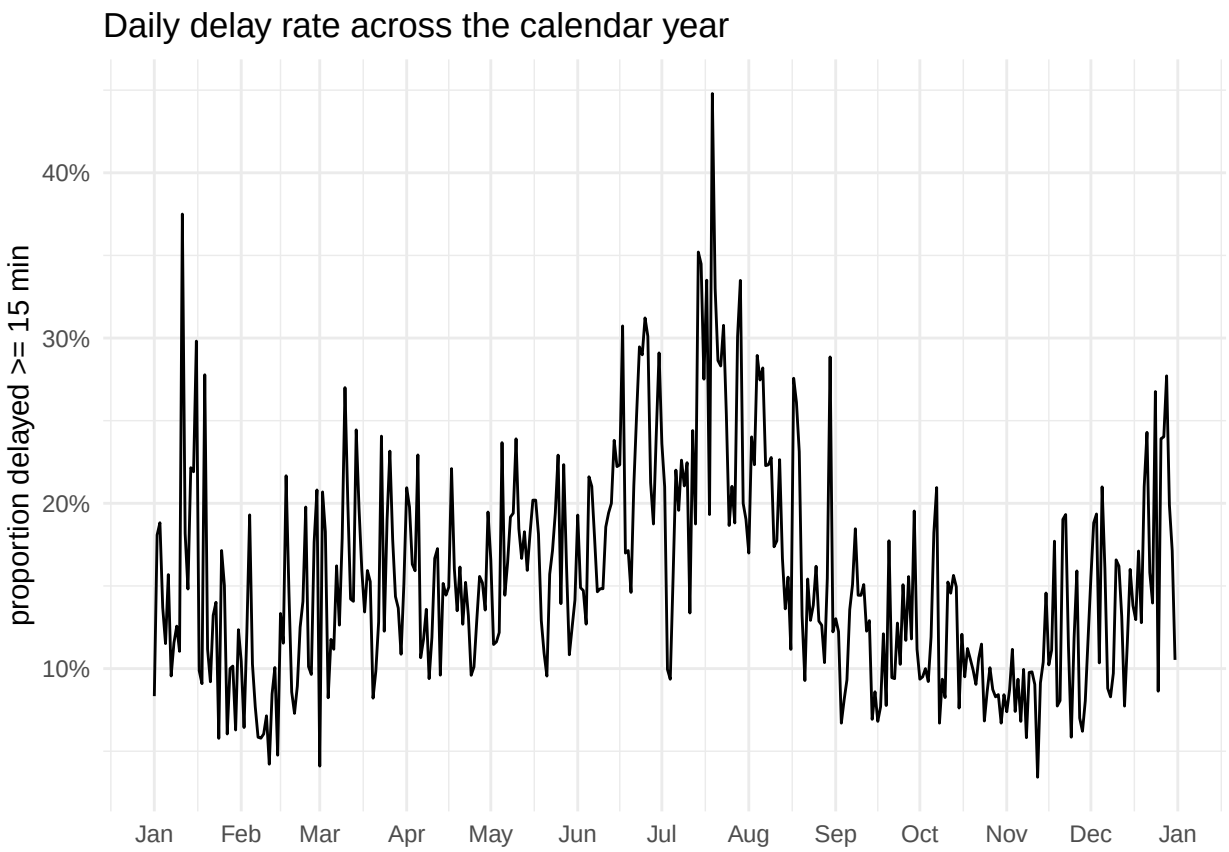



```
# Delay rate by day of week
train |>
 filter(!is.na(dep_del15)) |>
 group_by(day_of_week) |>
 summarise(
 delay_rate = mean(dep_del15),
 .groups = "drop"
 ) |>
 ggplot(aes(x = day_of_week, y = delay_rate, group = 1)) +
 geom_line() +
 geom_point() +
 scale_y_continuous(labels = percent_format()) +
 labs(
 title = "Delay rate by day of week",
```

```
 x = NULL,
 y = "proportion delayed >= 15 min"
 ) +
 theme_minimal()
```




```
# Delay rate by calendar date, aligned to a common year for display
train |>
 filter(!is.na(dep_del15)) |>
 mutate(plot_date = make_date(year = 2010, month = as.integer(as.character(month)), day = day_of_month)
 group_by(plot_date) |>
 summarise(
 delay_rate = mean(dep_del15),
 .groups = "drop"
 ) |>
 ggplot(aes(x = plot_date, y = delay_rate)) +
 geom_line() +
 scale_x_date(date_labels = "%b", date_breaks = "1 month") +
 scale_y_continuous(labels = percent_format()) +
 labs(
 title = "Daily delay rate across the calendar year",
 x = NULL,
 y = "proportion delayed >= 15 min"
 ) +
 theme_minimal()
```



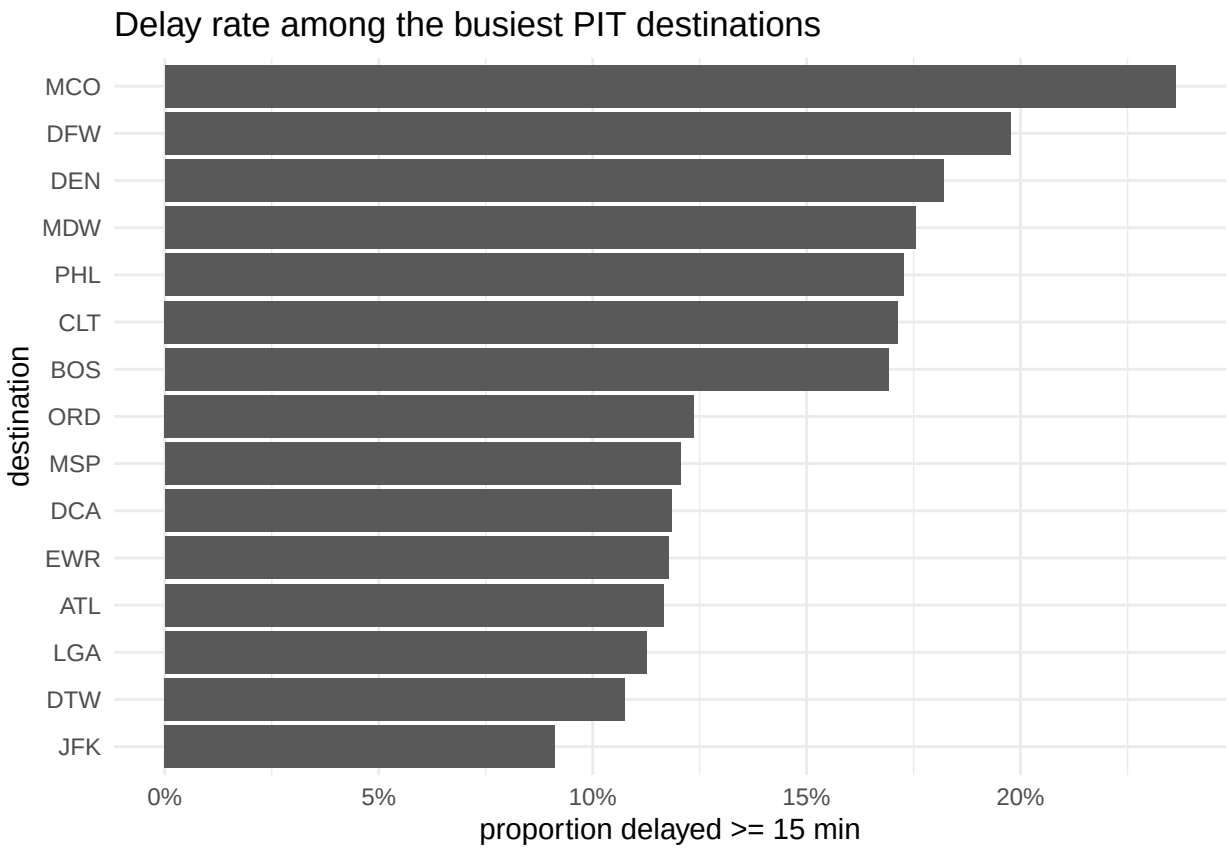
Delays are worse in the summer months and in December, which is consistent with heavier travel volume and more weather-related disruption. The day-of-week pattern is milder, but it still suggests that weekly operating patterns contain some signal. The daily calendar plot also shows that these seasonal peaks are not spread evenly across the month, which supports adding a simple calendar feature rather than relying only on month.

Destination and Distance Effects

```
top_dests <- train |>
 count(dest, sort = TRUE) |>
 slice_head(n = 15) |>
 pull(dest)

train |>
 filter(!is.na(dep_del15), dest %in% top_dests) |>
 group_by(dest) |>
 summarise(
 delay_rate = mean(dep_del15),
 n = n(),
 .groups = "drop"
 ) |>
 arrange(delay_rate) |>
 ggplot(aes(x = reorder(dest, delay_rate), y = delay_rate)) +
 geom_col() +
 coord_flip() +
```

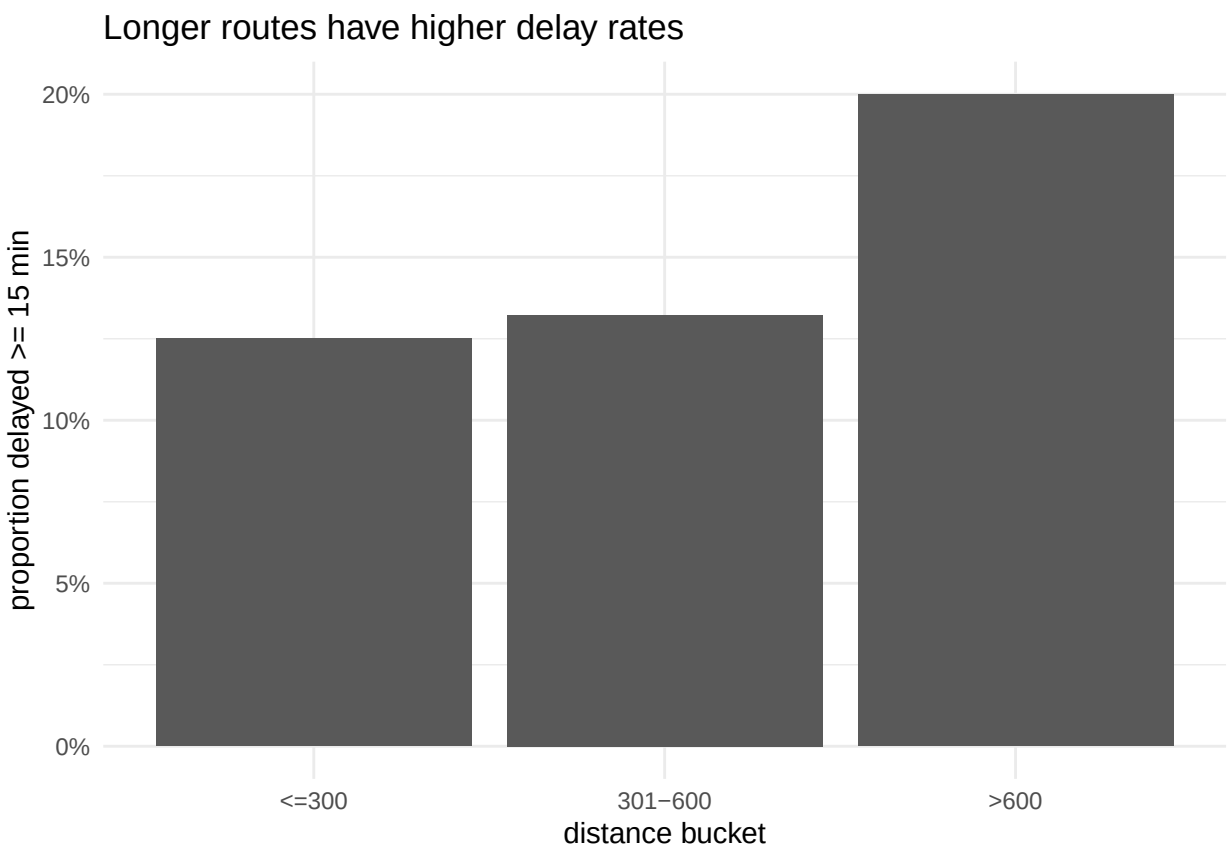
```
 scale_y_continuous(labels = percent_format()) +
 labs(
 title = "Delay rate among the busiest PIT destinations",
 x = "destination",
 y = "proportion delayed >= 15 min"
 ) +
 theme_minimal()
```




```
train <- train |>
 mutate(
 distance_bucket = case_when(
 distance > 600 ~ ">600",
 distance > 300 ~ "301-600",
 TRUE ~ "<=300"
 ),
 distance_bucket = factor(
 distance_bucket,
 levels = c("<=300", "301-600", ">600")
 )
 )

train |>
 filter(!is.na(dep_del15), !is.na(distance_bucket)) |>
 group_by(distance_bucket) |>
 summarise(
 delay_rate = mean(dep_del15),
```

```
n = n(),
.groups = "drop"
) |>
ggplot(aes(x = distance_bucket, y = delay_rate)) +
geom_col() +
scale_y_continuous(labels = percent_format()) +
labs(
title = "Longer routes have higher delay rates",
x = "distance bucket",
y = "proportion delayed >= 15 min"
) +
theme_minimal()
```



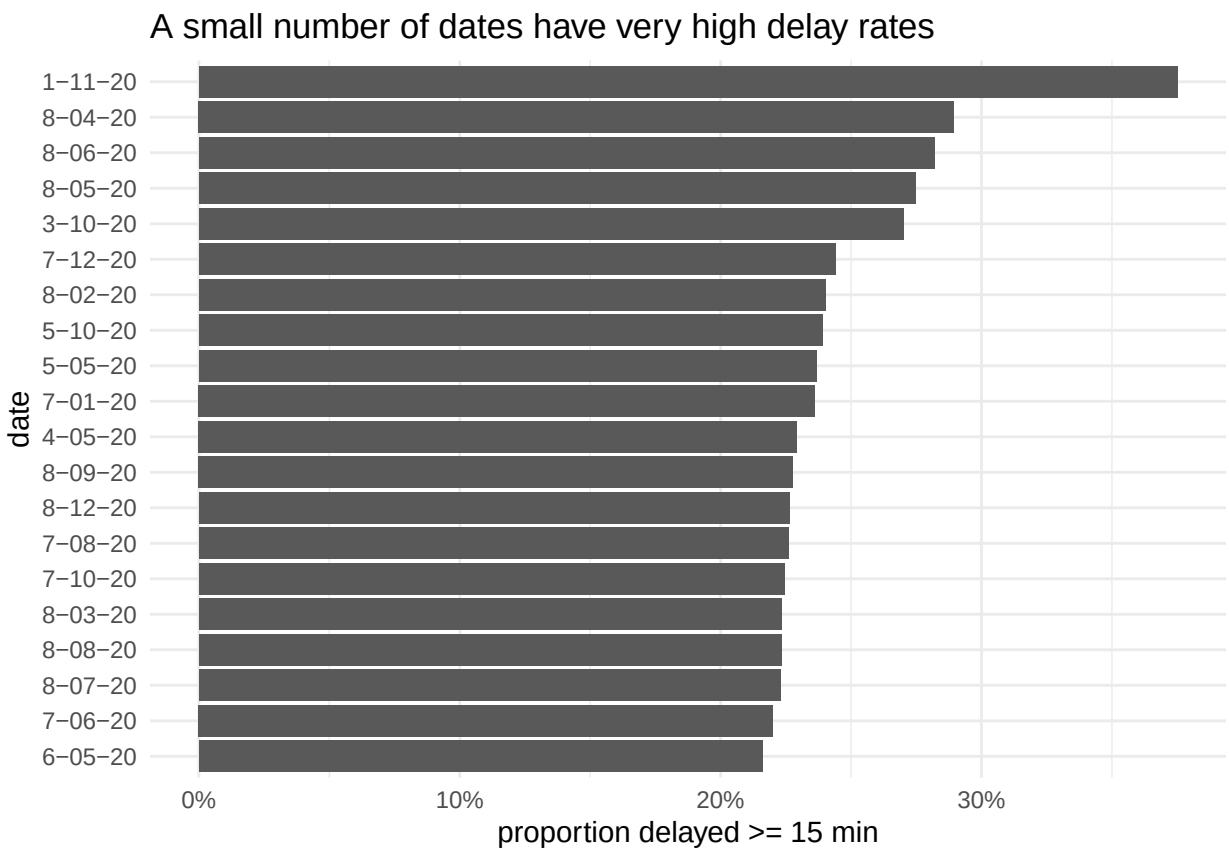
Delay rates differ across destinations, even among the busiest PIT routes, which suggests that route context adds information beyond carrier and time of day alone. Longer routes also show higher delay rates on average, so distance should remain in the model and may be easier to capture through both a raw continuous version and a bucketed factor version.

High-Delay Days

```
bad_days_tbl <- train |>
 filter(!is.na(dep_del15)) |>
 group_by(fl_date) |>
 summarise(
 delay_rate = mean(dep_del15),
```

```
  n = n(),
  .groups = "drop"
  ) |>
  filter(n >= 10) |>
  arrange(desc(delay_rate))

bad_days_tbl |>
  slice_head(n = 20) |>
  ggplot(aes(x = reorder(as.character(fl_date), delay_rate), y = delay_rate)) +
  geom_col() +
  coord_flip() +
  scale_y_continuous(labels = percent_format()) +
  labs(
  title = "A small number of dates have very high delay rates",
  x = "date",
  y = "proportion delayed >= 15 min"
  ) +
  theme_minimal()
```



A small number of days have unusually high delay rates, which suggests that there are broader operating shocks affecting many flights at once. That motivates adding dynamic same-day features based on earlier visible flights, along with a historical calendar-day disruption feature learned from the training data.

Feature Engineering

The EDA suggests that delay risk depends on a mix of stable historical tendencies and day-specific operating conditions. Based on those patterns, we engineer features that capture time-of-day effects, carrier and route history, route distance, seasonality, and same-day disruption.

Add Basic Modeling Features

```
add_basic_features <- function(df) {
 df |>
 mutate(
 peak_season = if_else(as.integer(as.character(month)) %in% c(6, 7, 8, 12), 1, 0),
 is_evening = if_else(sched_dep_hour >= 18, 1, 0),
 is_morning = if_else(sched_dep_hour <= 8, 1, 0),
 distance_bucket = case_when(
 distance > 600 ~ ">600",
 distance > 300 ~ "301-600",
 TRUE ~ "<=300"
 ),
 carrier_hour = paste(op_unique_carrier, sched_dep_hour, sep = "_"),
 # scheduled arrival hour at destination - late-night arrivals face thinner operations
 sched_arr_hour = floor(crs_arr_time / 100),
 is_late_arrival = if_else(sched_arr_hour >= 21 | sched_arr_hour <= 1, 1L, 0L),
 # scheduled flight duration - longer flights have more exposure to en-route disruption
 crs_elapsed_time = as.numeric(crs_elapsed_time)
 )
}

train <- add_basic_features(train)
test_visible <- add_basic_features(test_visible)
test_guess <- add_basic_features(test_guess)
```

These features directly reflect the strongest patterns from the EDA. In particular, `peak_season`, the morning and evening indicators, the distance bucket, and the carrier-by-hour combination all transform observed structure in the data into modeling variables.

Build Smoothed Historical Delay-Rate Features

```
global_delay_rate <- mean(train$dep_del15, na.rm = TRUE)

make_smoothed_rates <- function(df, group_var, target = "dep_del15", m = 100) {
 group_sym <- rlang::sym(group_var)
 target_sym <- rlang::sym(target)

 df |>
 filter(!is.na(!!target_sym)) |>
 group_by(!!group_sym) |>
 summarise(
 n = n(),
```

```r
 mean_target = mean(!!target_sym),
 .groups = "drop"
 ) |>
 mutate(
 smoothed_rate = (n * mean_target + m * global_delay_rate) / (n + m)
 ) |>
 select(!!group_sym, smoothed_rate, n)
}
```

```r
# Route = carrier + dest combination
train <- train |> mutate(route = paste(op_unique_carrier, dest, sep = "_"))
test_visible <- test_visible |> mutate(route = paste(op_unique_carrier, dest, sep = "_"))
test_guess <- test_guess |> mutate(route = paste(op_unique_carrier, dest, sep = "_"))

# Carrier and dest rates computed on full train - stable enough that leakage is minimal
carrier_rates <- make_smoothed_rates(train, "op_unique_carrier", m = 200) |>
 rename(carrier_delay_rate_smooth = smoothed_rate, carrier_n = n)

dest_rates <- make_smoothed_rates(train, "dest", m = 200) |>
 rename(dest_delay_rate_smooth = smoothed_rate, dest_n = n)
```

```r
# OOF (out-of-fold) smoothed rates for important features
# Each row's rate is computed from the other 4 folds only, preventing label leakage
set.seed(100)
n_folds <- 5
fold_id <- sample(rep(1:n_folds, length.out = nrow(train)))

make_oof_rates <- function(df, group_var, target = "dep_del15", m = 100) {
  group_sym <- rlang::sym(group_var)
  target_sym <- rlang::sym(target)
  out_col <- paste0(group_var, "_oof_rate")
  global <- mean(df[[target]], na.rm = TRUE)

  result <- numeric(nrow(df))
  for (k in 1:n_folds) {
    in_fold <- which(fold_id == k)
    out_fold <- which(fold_id != k)
    lookup <- df[out_fold, ] |>
      filter(!is.na(!!target_sym)) |>
      group_by(!!group_sym) |>
      summarise(n = n(), mean_t = mean(!!target_sym), .groups = "drop") |>
      mutate(rate = (n * mean_t + m * global) / (n + m))
    merged <- df[in_fold, ] |>
      select(!!group_sym) |>
      left_join(lookup |> select(!!group_sym, rate), by = group_var)
    result[in_fold] <- coalesce(merged$rate, global)
  }
  result
}

# Use stronger smoothing (bigger m) for route and flight level features bc those groups
# are more granular and have fewer observations per group (noisier raw delay rates)
train$carrier_hour_delay_rate_smooth <- make_oof_rates(train, "carrier_hour", m = 100)
```

```r
train$route_delay_rate_smooth <- make_oof_rates(train, "route", m = 150)

# Week-of-year rate - replaces raw calendar-day rate (~2 obs/day with 2 years of data)
# Week of year gives ~14 observations per cell, same seasonal signal but more stable
train <- train |> mutate(week_of_year = as.character(week(fl_date)))
test_visible <- test_visible |> mutate(week_of_year = as.character(week(fl_date)))
test_guess <- test_guess |> mutate(week_of_year = as.character(week(fl_date)))
train$week_delay_rate_smooth <- make_oof_rates(train, "week_of_year", m = 50)
train$avg_delay_rate_for_calendar_day <- train$week_delay_rate_smooth  # keep for computing high_disrup

# For test sets, use the full-train lookup
carrier_hour_rates <- make_smoothed_rates(train, "carrier_hour", m = 100) |>
  rename(carrier_hour_delay_rate_smooth = smoothed_rate, carrier_hour_n = n)

route_rates <- make_smoothed_rates(train, "route", m = 150) |>
  rename(route_delay_rate_smooth = smoothed_rate, route_n = n)

week_rates <- make_smoothed_rates(train, "week_of_year", m = 50) |>
  rename(week_delay_rate_smooth = smoothed_rate, week_n = n)

bad_day_lookup <- train |>
  filter(!is.na(dep_del15)) |>
  group_by(week_of_year) |>
  summarise(avg_delay_rate_for_calendar_day = mean(dep_del15), .groups = "drop")
```

Rather than using raw historical means, we shrink each group toward the global average when the number of observations is small. For fine-grained features like carrier-hour and route, we use out-of-fold encoding so that each training row's smoothed rate is computed from held-out folds only, preventing label leakage into the validation set. Test sets always use full-train lookups since their labels are unseen.

Hourly Congestion at PIT

```r
# Number of flights scheduled out of PIT in the same departure hour
hourly_congestion <- train |>
  group_by(fl_date, sched_dep_hour) |>
  summarise(flights_same_hour = n(), .groups = "drop")

train <- train |>
  left_join(hourly_congestion, by = c("fl_date", "sched_dep_hour"))

# For test sets, use average congestion by hour across training days
# (bc we don't know actual 2025 schedules ahead of time for the guess flights)
avg_hourly_congestion <- train |>
  group_by(sched_dep_hour) |>
  summarise(avg_flights_same_hour = mean(flights_same_hour, na.rm = TRUE), .groups = "drop")

test_visible <- test_visible |>
  left_join(
    test_visible |>
      group_by(fl_date, sched_dep_hour) |>
      summarise(flights_same_hour = n(), .groups = "drop"),
```

```r
    by = c("fl_date", "sched_dep_hour")
  )

test_guess <- test_guess |>
  left_join(avg_hourly_congestion, by = "sched_dep_hour") |>
  mutate(flights_same_hour = avg_flights_same_hour)
```

Apply Historical Features to Train and Test

```r
# For train OOF columns already attached above so only need to join carrier/dest rates
# For test sets join all lookup tables from full training data
apply_rate_features <- function(df, is_train = FALSE) {
  df <- df |>
    left_join(carrier_rates, by = "op_unique_carrier") |>
    left_join(dest_rates, by = "dest") |>
    mutate(
      carrier_delay_rate_smooth = coalesce(carrier_delay_rate_smooth, global_delay_rate),
      dest_delay_rate_smooth = coalesce(dest_delay_rate_smooth, global_delay_rate)
    )
  if (!is_train) {
    df <- df |>
      left_join(carrier_hour_rates, by = "carrier_hour") |>
      left_join(route_rates, by = "route") |>
      left_join(week_rates, by = "week_of_year") |>
      left_join(bad_day_lookup, by = "week_of_year") |>
      mutate(
        carrier_hour_delay_rate_smooth = coalesce(carrier_hour_delay_rate_smooth, global_delay_rate),
        route_delay_rate_smooth = coalesce(route_delay_rate_smooth, global_delay_rate),
        week_delay_rate_smooth = coalesce(week_delay_rate_smooth, global_delay_rate),
        avg_delay_rate_for_calendar_day = coalesce(avg_delay_rate_for_calendar_day, global_delay_rate)
      )
  }
  df
}

train_fe <- apply_rate_features(train, is_train = TRUE)
test_visible_fe <- apply_rate_features(test_visible, is_train = FALSE)
test_guess_fe <- apply_rate_features(test_guess, is_train = FALSE)
```

All of these encodings are learned using the training data and then applied to the test sets. If a group in the test set was not seen in training, we fall back to the global delay rate.

Add Running Same-Day Delay Features

```r
add_running_day_features <- function(df) {
 df |>
 arrange(fl_date, crs_dep_time) |>
 group_by(fl_date) |>
 mutate(
```

```r
 before_today = row_number() - 1,
 delays_before_today = lag(cumsum(replace_na(dep_del15, 0)), default = 0),
 prior_same_day_delay_rate = if_else(
 before_today > 0,
 delays_before_today / before_today,
 0
 )
 ) |>
 ungroup()
}

train_fe <- add_running_day_features(train_fe)
test_visible_fe <- add_running_day_features(test_visible_fe)
test_guess_fe <- add_running_day_features(test_guess_fe)
```

This feature captures how disrupted the system already is earlier in the day. It might be especially useful because the prediction setup provides visibility into earlier flights on the same day.

```r
# Rich same-day context from visible flights
# The project allows using all visible-window flights to predict guess flights.
# We build summaries from both departures and arrivals in the visible window.

# Overall departure health for the day
vis_dep_summary <- test_visible_fe |>
  filter(!is.na(dep_del15)) |>
  arrange(fl_date, crs_dep_time) |>
  group_by(fl_date) |>
  summarise(
    visible_count = n(),
    visible_delay_rate = mean(dep_del15),
    visible_max_delay = max(dep_delay, na.rm = TRUE),  # flags big disruption events
    visible_n_delayed = sum(dep_del15),
    visible_recent_rate = mean(tail(dep_del15, 5)),  # last 5 flights, more current signal
    .groups = "drop"
  ) |>
  mutate(
    # a single delay > 60 min often signals a ground stop or weather event
    visible_severe_delay = if_else(visible_max_delay > 60, 1L, 0L),
    visible_max_delay = if_else(is.infinite(visible_max_delay), 0, visible_max_delay)
  )

# Per-carrier delay rate from visible flights that day
# e.g. if United has been running 40 min late all morning, that predicts the 7pm United flight
vis_carrier_summary <- test_visible_fe |>
  filter(!is.na(dep_del15)) |>
  group_by(fl_date, op_unique_carrier) |>
  summarise(
    visible_carrier_delay_rate = mean(dep_del15),
    visible_carrier_n = n(),
    .groups = "drop"
  )

# Arrival health at PIT from visible window
```

```r
# planes arriving late become planes departing late
arrivals_vis_summary <- arrivals_test_visible |>
  filter(!is.na(arr_delay)) |>
  group_by(fl_date) |>
  summarise(
    vis_arr_delay_rate = mean(arr_del15, na.rm = TRUE),  # fraction of arrivals that were late
    vis_arr_mean_delay = mean(arr_delay, na.rm = TRUE),  # mean arrival delay in minutes
    .groups = "drop"
  )

# Join all summaries onto guess set
test_guess_fe <- test_guess_fe |>
  left_join(vis_dep_summary, by = "fl_date") |>
  left_join(vis_carrier_summary, by = c("fl_date", "op_unique_carrier")) |>
  left_join(arrivals_vis_summary, by = "fl_date") |>
  mutate(
    visible_count = coalesce(visible_count, 0L),
    visible_delay_rate = coalesce(visible_delay_rate, global_delay_rate),
    visible_max_delay = coalesce(visible_max_delay, 0),
    visible_n_delayed = coalesce(visible_n_delayed, 0L),
    visible_recent_rate = coalesce(visible_recent_rate, global_delay_rate),
    visible_severe_delay = coalesce(visible_severe_delay, 0L),
    visible_carrier_delay_rate = coalesce(visible_carrier_delay_rate, visible_delay_rate),
    visible_carrier_n = coalesce(visible_carrier_n, 0L),
    vis_arr_delay_rate = coalesce(vis_arr_delay_rate, global_delay_rate),
    vis_arr_mean_delay = coalesce(vis_arr_mean_delay, 0)
  )

add_running_visible_features <- function(df) {
  df |>
    arrange(fl_date, crs_dep_time) |>
    group_by(fl_date) |>
    mutate(
      visible_count = row_number() - 1,
      visible_n_delayed = lag(cumsum(replace_na(dep_del15, 0)), default = 0),
      visible_delay_rate = if_else(
        visible_count > 0,
        visible_n_delayed / visible_count,
        global_delay_rate
      )
    ) |>
    mutate(
      visible_recent_rate = purrr::map_dbl(
        row_number(),
        \(i) {
          if (i == 1) return(global_delay_rate)
          start_i <- max(1, i - 5)
          mean(replace_na(dep_del15[start_i:(i - 1)], 0))
        }
      )
    ) |>
    mutate(
      visible_max_delay = purrr::map_dbl(
```

```
        row_number(),
        \(i) {
          if (i == 1) return(0)
          start_i <- 1
          vals <- dep_delay[start_i:(i - 1)]
          vals <- vals[!is.na(vals)]
          if (length(vals) == 0) 0 else max(vals)
        }
      ),
      visible_severe_delay = if_else(visible_max_delay > 60, 1L, 0L)
    ) |>
    ungroup() |>
    group_by(fl_date, op_unique_carrier) |>
    arrange(crs_dep_time, .by_group = TRUE) |>
    mutate(
      visible_carrier_n = row_number() - 1,
      visible_carrier_n_delayed = lag(cumsum(replace_na(dep_del15, 0)), default = 0),
      visible_carrier_delay_rate = if_else(
        visible_carrier_n > 0,
        visible_carrier_n_delayed / visible_carrier_n,
        visible_delay_rate
      )
    ) |>
    ungroup() |>
    select(-visible_carrier_n_delayed)
}

train_fe <- add_running_visible_features(train_fe)
test_visible_fe <- add_running_visible_features(test_visible_fe)
```

For the guess set we can't use the unknown future labels directly. Instead, we summarize the flights that are already visible on that date — both departures with observed delays and arrivals at PIT — and merge those daily summaries into the flights we need to predict.

Carrier and Destination Encodings

Also, the raw `op_unique_carrier` and `dest` factors can't be used directly because 2025 may contain carriers or destinations not seen in 2023-2024 training data, which would produce NA columns when building the model matrix. We already have smoothed delay rates for both, which solve this problem. We add a few additional encodings below to recover some of the signal that raw dummies would have provided.

```
# Option 1: carrier size group
# Collapses carriers by volume at PIT so new/rare carriers in 2025 fall into
# a meaningful bucket rather than being dropped or NA
carrier_size <- train |>
  count(op_unique_carrier) |>
  mutate(carrier_size = case_when(
    n >= 5000 ~ "large",
    n >= 1000 ~ "medium",
    TRUE ~ "small"
  )) |>
  select(op_unique_carrier, carrier_size)
```

```r
# Option 2: hub vs. non-hub destination
# Major hub airports tend to have higher system-wide congestion; captures that
# without needing a dummy for every destination
hub_airports <- c("ATL", "ORD", "DFW", "DEN", "CLT", "JFK", "LAX",
                  "LGA", "EWR", "SFO", "SEA", "MIA", "BOS", "MSP", "DTW", "PHL")

# Option 3: regional grouping for destination
# Groups destinations by geography - more stable across years than individual airports
dest_region <- train |>
  distinct(dest) |>
  mutate(dest_region = case_when(
    dest %in% c("BOS", "JFK", "LGA", "EWR", "PHL", "DCA", "IAD", "BWI") ~ "northeast",
    dest %in% c("ATL", "CLT", "MIA", "TPA", "MCO", "RDU", "BNA", "MEM") ~ "south",
    dest %in% c("ORD", "MDW", "DTW", "MSP", "CLE", "CMH", "IND", "MKE") ~ "midwest",
    dest %in% c("DFW", "DAL", "IAH", "HOU", "MSY", "SAT", "AUS") ~ "southwest",
    dest %in% c("DEN", "SLC", "PHX", "LAS", "ABQ") ~ "mountain",
    dest %in% c("LAX", "SFO", "SEA", "PDX", "SAN", "SJC", "OAK") ~ "west",
    TRUE ~ "other"
  ))

add_carrier_dest_features <- function(df) {
  df |>
    left_join(carrier_size, by = "op_unique_carrier") |>
    left_join(dest_region, by = "dest") |>
    mutate(
      carrier_size = coalesce(carrier_size, "small"),  # unseen carriers are small
      dest_region = coalesce(dest_region, "other"),  # unseen dests are other
      is_hub_dest = if_else(dest %in% hub_airports, 1L, 0L),
      carrier_size = factor(carrier_size, levels = c("small", "medium", "large")),
      dest_region = factor(dest_region),
      # binary flag derived from week rate
      high_disruption_calendar_day = if_else(
        week_delay_rate_smooth >= quantile(week_delay_rate_smooth, 0.90, na.rm = TRUE),
        1L, 0L
      )
    )
}

train_fe <- add_carrier_dest_features(train_fe)
test_visible_fe <- add_carrier_dest_features(test_visible_fe)
test_guess_fe <- add_carrier_dest_features(test_guess_fe)
```

The smoothed delay rates (`carrier_delay_rate_smooth`, `dest_delay_rate_smooth`) already capture most of the signal from the raw factors — they encode historical delay tendency per group and fall back to the global mean for unseen levels. The new features above add complementary structure: carrier size captures operational scale, hub destination captures congestion exposure, and regional grouping adds geographic context that's stable even if specific airports weren't in training.

Final Modeling Data Frames

```r
model_vars <- c(
  "dep_del15",
  "sched_dep_hour",
  "sched_arr_hour",
  "is_late_arrival",
  "crs_elapsed_time",
  "carrier_size",
  "dest_region",
  "is_hub_dest",
  "distance",
  "distance_bucket",
  "month",
  "day_of_week",
  "peak_season",
  "is_evening",
  "is_morning",
  "carrier_delay_rate_smooth",
  "dest_delay_rate_smooth",
  "route_delay_rate_smooth",
  "prior_same_day_delay_rate",
  "before_today",
  "visible_delay_rate",
  "visible_count",
  "visible_max_delay",
  "visible_n_delayed",
  "visible_recent_rate",
  "visible_severe_delay",
  "visible_carrier_delay_rate",
  "visible_carrier_n",
  "flights_same_hour",
  "high_disruption_calendar_day"
)

train_model_df <- train_fe |>
  select(any_of(model_vars)) |>
  filter(!is.na(dep_del15))

test_visible_model_df <- test_visible_fe |>
  select(any_of(model_vars))

test_guess_model_df <- test_guess_fe |>
  select(any_of(model_vars))
```

At this point, `train_model_df` contains the response and engineered predictors for model fitting, while `test_guess_model_df` contains the same predictors for the flights we need to evaluate. The final feature set is based on the main conclusions from the EDA: time of day is a major driver, carrier, destination, route are important, and we believe same-day disruption adds useful information.

Modelling

Setup

```r
# Aligns model matrices between train and test so column sets always match
# If a factor level exists in train but not in test, adds a zero column
# If test has extra columns not in train, drop
# Prevents dimension mismatches when new factor levels appear in 2025 data
make_aligned_matrix <- function(formula, train_df, test_df) {
  x_train <- model.matrix(formula, data = train_df)[, -1, drop = FALSE]
  x_test <- model.matrix(formula, data = test_df)[, -1, drop = FALSE]

  missing_cols <- setdiff(colnames(x_train), colnames(x_test))
  if (length(missing_cols) > 0) {
    for (col in missing_cols) {
      x_test <- cbind(x_test, 0)
      colnames(x_test)[ncol(x_test)] <- col
    }
  }

  extra_cols <- setdiff(colnames(x_test), colnames(x_train))
  if (length(extra_cols) > 0) {
    x_test <- x_test[, !(colnames(x_test) %in% extra_cols), drop = FALSE]
  }

  x_test <- x_test[, colnames(x_train), drop = FALSE]

  list(
    x_train = x_train,
    x_test = x_test
  )
}
```

```r
# 80/20 train/validation split
set.seed(100)

n <- nrow(train_model_df)
train_idx <- sample(1:n, size = 0.8 * n)

train_split <- train_model_df[train_idx, ]
test_split <- train_model_df[-train_idx, ]
```

```r
# function to track AUC and accuracy, appends one row per model to a running results table
compute_auc <- function(auc_df = NULL, train_y, train_prob,
                        test_y, test_prob, model_name) {
  auc_train <- roc(train_y, train_prob) |> auc()
  auc_test <- roc(test_y, test_prob) |> auc()

  accur_train_0.5 <- mean(train_y == (train_prob > 0.5))
  accur_test_0.5 <- mean(test_y == (test_prob > 0.5))

  accur_train_0.6 <- mean(train_y == (train_prob > 0.6))
```

```r
  accur_test_0.6 <- mean(test_y == (test_prob > 0.6))

  df <- data.frame(
    model_name,
    auc_train,
    auc_test,
    accur_train_0.5,
    accur_test_0.5,
    accur_train_0.6,
    accur_test_0.6
  )

  if (is.null(auc_df)) {
    return(df)
  } else {
    return(rbind(auc_df, df))
  }
}
```

```r
y_train <- train_split$dep_del15
y_test <- test_split$dep_del15

# For our linear models (logistic regression, LASSO): add a quadratic hour term
# to try and capture the nonlinear delay-by-hour relationship
lm_mats <- make_aligned_matrix(
  dep_del15 ~ . + poly(sched_dep_hour, 2),
  train_split,
  test_split
)

x_train_lm <- lm_mats$x_train
x_test_lm <- lm_mats$x_test

# XGBoost: fit using raw features only
xgb_mats <- make_aligned_matrix(
  dep_del15 ~ .,
  train_split,
  test_split
)

x_train_xgb <- xgb_mats$x_train
x_test_xgb <- xgb_mats$x_test

# Full training set matrix used for XGBoost CV hyperparameter tuning
# Tuning on the full train rather than the 80/20 split --> more stable estimates
x_full_xgb <- model.matrix(dep_del15 ~ ., data = train_model_df)[, -1, drop = FALSE]
y_full <- train_model_df$dep_del15
```

Logistic Regression

```r
# Baseline log reg model; benchmark
log_reg <- glm(
  dep_del15 ~ . + poly(sched_dep_hour, 2),
  family = "binomial",
  data = train_split
)

train_prob <- predict(log_reg, type = "response", newdata = train_split)
test_prob <- predict(log_reg, type = "response", newdata = test_split)

auc_df <- compute_auc(
  train_y = train_split$dep_del15,
  train_prob = train_prob,
  test_y = test_split$dep_del15,
  test_prob = test_prob,
  model_name = "Log Reg"
)
```

LASSO

```r
# Logistic regression with L1 penalty, use cv to choose lambda
cv_lasso <- cv.glmnet(
  x_train_lm,
  y_train,
  family = "binomial",
  alpha = 1
)

train_prob <- predict(
  cv_lasso,
  newx = x_train_lm,
  s = "lambda.min",
  type = "response"
) |>
  as.vector()

test_prob <- predict(
  cv_lasso,
  newx = x_test_lm,
  s = "lambda.min",
  type = "response"
) |>
  as.vector()

auc_df <- compute_auc(
  auc_df = auc_df,
  train_y = y_train,
  train_prob = train_prob,
  test_y = y_test,
  test_prob = test_prob,
  model_name = "Log Reg (LASSO)"
```

```
)
```

XGBoost (tuned)

XGBoost builds an ensemble of shallow decision trees sequentially, each correcting the errors of the previous one. It can handle the nonlinear relationships and feature interactions automatically, so it should outperform the previous two linear models.

We tune over a grid of key hyperparameters: `max_depth` (tree complexity), `eta` (learning rate / shrinkage per tree), `subsample` and `colsample_bytree` (row/column subsampling to reduce overfitting), `min_child_weight` (minimum observations per leaf, which prevents splits on small noisy groups), and `gamma` (minimum gain required to make a split). `nrounds` is selected by early stopping against 5-fold CV AUC on the full training set.

```
dtrain <- xgb.DMatrix(data = x_train_xgb, label = y_train)
dtest <- xgb.DMatrix(data = x_test_xgb, label = y_test)
dfull <- xgb.DMatrix(data = x_full_xgb, label = y_full)

xgb_grid <- expand.grid(
  max_depth = c(3, 4, 5),
  eta = c(0.05, 0.1),
  subsample = c(0.7, 0.9),
  colsample_bytree = c(0.7, 0.9),
  min_child_weight = c(10, 30),
  gamma = c(0, 1)
)

best_auc <- 0
best_params <- NULL
best_nrounds <- 100

for (i in seq_len(nrow(xgb_grid))) {
  params_i <- list(
    objective = "binary:logistic",
    eval_metric = "auc",
    max_depth = xgb_grid$max_depth[i],
    eta = xgb_grid$eta[i],
    subsample = xgb_grid$subsample[i],
    colsample_bytree = xgb_grid$colsample_bytree[i],
    min_child_weight = xgb_grid$min_child_weight[i],
    gamma = xgb_grid$gamma[i]
  )

  cv_result <- xgb.cv(
    params = params_i,
    data = dfull,
    nrounds = 500,
    nfold = 5,
    early_stopping_rounds = 30,
    verbose = 0
  )

  nrounds_i <- cv_result$best_iteration
```

```r
  if (is.null(nrounds_i) || length(nrounds_i) == 0) {
    nrounds_i <- which.max(cv_result$evaluation_log$test_auc_mean)
  }

  best_i <- cv_result$evaluation_log$test_auc_mean[nrounds_i]

  if (best_i > best_auc) {
    best_auc <- best_i
    best_params <- params_i
    best_nrounds <- nrounds_i
  }
}

cat("Best CV AUC:", round(best_auc, 4), "| nrounds:", best_nrounds, "\n")
```

```
## Best CV AUC: 0.7443 | nrounds: 251
```

```r
xgb_model <- xgb.train(
  params = best_params,
  data = dfull,
  nrounds = best_nrounds,
  verbose = 0
)

train_prob <- predict(xgb_model, dtrain)
test_prob <- predict(xgb_model, dtest)

auc_df <- compute_auc(
  auc_df = auc_df,
  train_y = y_train,
  train_prob = train_prob,
  test_y = y_test,
  test_prob = test_prob,
  model_name = "XGBoost (tuned)"
)
```

```r
# Function that ensures factor columns in new_df match train_df,
# and coerces numeric/integer types to match
# prevents errors when 2025 data has factor levels not in 2023-2024 training
align_to_training <- function(new_df, train_df) {
  common_cols <- intersect(names(train_df), names(new_df))

  for (col in common_cols) {
    # align factor columns
    if (is.factor(train_df[[col]])) {
      new_df[[col]] <- factor(as.character(new_df[[col]]),
                              levels = levels(train_df[[col]]))
    }

    # align numeric/integer columns
    if (is.numeric(train_df[[col]]) && !is.numeric(new_df[[col]])) {
      new_df[[col]] <- as.numeric(new_df[[col]])
    }
```

```
    if (is.integer(train_df[[col]]) && !is.integer(new_df[[col]])) {
      new_df[[col]] <- as.integer(new_df[[col]])
    }
  }

  new_df
}
```

Random Forest

```
# Ensemble of decision trees trained on bootstrap samples with random feature subsets.
# More stable than a single tree and naturally handles nonlinearity and interactions.
# dep_del15 cast to factor so randomForest treats this as classification (not regression).
train_split_rf <- train_split |>
  mutate(dep_del15 = as.factor(dep_del15))

test_split_rf <- test_split |>
  mutate(dep_del15 = as.factor(dep_del15))

rf_model <- randomForest(
  dep_del15 ~ .,
  data = train_split_rf,
  ntree = 300,
  importance = TRUE
)

train_prob <- predict(rf_model, newdata = train_split_rf, type = "prob")[, "1"]

test_split_rf_aligned <- align_to_training(test_split_rf, train_split_rf)

test_prob <- predict(rf_model, newdata = test_split_rf_aligned, type = "prob")[, "1"]

auc_df <- compute_auc(
  auc_df = auc_df,
  train_y = y_train,
  train_prob = train_prob,
  test_y = y_test,
  test_prob = test_prob,
  model_name = "Random Forest"
)
```

Balanced Random Forest

```
# Same as Random Forest but with inverse-frequency class weights
# Increases weight for the delayed class (imbalanced ~20%) so the model doesn't just learn
# to predict "on time" for everything, which would give high accuracy but miss delays
class_counts <- table(train_split_rf$dep_del15)

class_weights <- as.numeric(
```

```r
  sum(class_counts) / (length(class_counts) * class_counts)
)
names(class_weights) <- names(class_counts)

brf_model <- randomForest(
  dep_del15 ~ .,
  data = train_split_rf,
  ntree = 300,
  classwt = class_weights
)

train_prob <- predict(brf_model, newdata = train_split_rf, type = "prob")[, "1"]

test_split_rf_aligned <- align_to_training(test_split_rf, train_split_rf)

test_prob <- predict(brf_model, newdata = test_split_rf_aligned, type = "prob")[, "1"]

auc_df <- compute_auc(
  auc_df = auc_df,
  train_y = y_train,
  train_prob = train_prob,
  test_y = y_test,
  test_prob = test_prob,
  model_name = "Balanced RF"
)
```

Ensembles

We compare: (1) the naive average and (2) a weighted average that upweights XGBoost and RF. Balanced RF is excluded from the weighted ensemble — class weighting distorts its probability estimates enough that including it hurts AUC despite improving raw accuracy.

```r
preds_df <- data.frame(
  y_true = as.numeric(as.character(test_split$dep_del15)),
  log_reg = predict(log_reg, type = "response", newdata = test_split),
  lasso = predict(cv_lasso, newx = x_test_lm, s = "lambda.min", type = "response") |> as.vector(),
  xgboost = predict(xgb_model, dtest),
  rf = predict(rf_model, newdata = test_split_rf, type = "prob")[, "1"],
  brf = predict(brf_model, newdata = test_split_rf, type = "prob")[, "1"]
)

# Simple average
preds_df$avg_prob <- rowMeans(preds_df[, c("log_reg", "lasso", "xgboost", "rf", "brf")])

auc_df <- compute_auc(
  auc_df = auc_df,
  train_y = preds_df$y_true,
  train_prob = preds_df$avg_prob,
  test_y = preds_df$y_true,
  test_prob = preds_df$avg_prob,
  model_name = "Ensemble (Avg)"
)
```

```r
# Weighted average - BRF excluded bc it's bad in terms of AUC
preds_df$weighted_prob <- with(
  preds_df,
  0.55 * xgboost +
    0.20 * rf +
    0.15 * lasso +
    0.10 * log_reg
)

auc_df <- compute_auc(
  auc_df = auc_df,
  train_y = preds_df$y_true,
  train_prob = preds_df$weighted_prob,
  test_y = preds_df$y_true,
  test_prob = preds_df$weighted_prob,
  model_name = "Ensemble (Weighted)"
)

# Majority vote
pred_class <- preds_df[, c("log_reg", "lasso", "xgboost", "rf", "brf")] > 0.5
preds_df$majority_prob <- rowMeans(pred_class)

auc_df <- compute_auc(
  auc_df = auc_df,
  train_y = preds_df$y_true,
  train_prob = preds_df$majority_prob,
  test_y = preds_df$y_true,
  test_prob = preds_df$majority_prob,
  model_name = "Ensemble (Majority Vote)"
)
```

Results - 2025 Visible

```r
# Evaluate each model on the 2025 visible set, which has flights we can observe
# and therefore acts as our proxy for true test performance before submitting on the guess set
compute_vis_auc <- function(auc_df = NULL, y, prob, model_name) {
  auc <- roc(y, prob) |> auc()
  accuracy <- mean(y == (prob > 0.5), na.rm = TRUE)

  df <- data.frame(model_name, auc, accuracy)

  if (is.null(auc_df)) {
    return(df)
  } else {
    return(rbind(auc_df, df))
  }
}
```

```r
test_visible_model_df_eval <- test_visible_model_df |>
  filter(!is.na(dep_del15))
```

```r
y_true <- as.numeric(as.character(test_visible_model_df_eval$dep_del15))
y_vis <- test_visible_model_df_eval$dep_del15

# Build aligned matrices using the original training split as the reference
vis_lm_mats <- make_aligned_matrix(
  dep_del15 ~ . + poly(sched_dep_hour, 2),
  train_split,
  test_visible_model_df_eval
)

x_vis_lm <- vis_lm_mats$x_test

vis_xgb_mats <- make_aligned_matrix(
  dep_del15 ~ .,
  train_split,
  test_visible_model_df_eval
)

x_vis_xgb <- vis_xgb_mats$x_test
dvis <- xgb.DMatrix(data = x_vis_xgb, label = y_vis)

# Logistic / lasso / xgboost probabilities
log_reg_prob <- predict(log_reg, type = "response", newdata = test_visible_model_df_eval)

lasso_prob <- predict(
  cv_lasso,
  newx = x_vis_lm,
  s = "lambda.min",
  type = "response"
) |>
  as.vector()

xgb_prob <- predict(xgb_model, dvis)

# RF-compatible visible data
test_visible_rf <- test_visible_model_df_eval |>
  mutate(dep_del15 = as.factor(dep_del15))

test_visible_rf <- align_to_training(test_visible_rf, train_split_rf)

rf_prob <- predict(
  rf_model,
  newdata = test_visible_rf,
  type = "prob"
)[, "1"]

brf_prob <- predict(
  brf_model,
  newdata = test_visible_rf,
  type = "prob"
)[, "1"]

avg_prob <- rowMeans(data.frame(log_reg_prob, lasso_prob, xgb_prob, rf_prob, brf_prob))
```

```r
# BRF excluded from weighted ensemble, not good
weighted_prob <- 0.55 * xgb_prob +
  0.20 * rf_prob +
  0.15 * lasso_prob +
  0.10 * log_reg_prob

vis_auc <- compute_vis_auc(
  y = y_true,
  prob = log_reg_prob,
  model_name = "Log Reg"
)

vis_auc <- compute_vis_auc(
  vis_auc,
  y = y_true,
  prob = lasso_prob,
  model_name = "LASSO"
)

vis_auc <- compute_vis_auc(
  vis_auc,
  y = y_true,
  prob = xgb_prob,
  model_name = "XGBoost (tuned)"
)

vis_auc <- compute_vis_auc(
  vis_auc,
  y = y_true,
  prob = rf_prob,
  model_name = "RF"
)

vis_auc <- compute_vis_auc(
  vis_auc,
  y = y_true,
  prob = brf_prob,
  model_name = "Balanced RF"
)

vis_auc <- compute_vis_auc(
  vis_auc,
  y = y_true,
  prob = avg_prob,
  model_name = "Ensemble (Avg)"
)

vis_auc <- compute_vis_auc(
  vis_auc,
  y = y_true,
  prob = weighted_prob,
  model_name = "Ensemble (Weighted)"
)
```

```r
vis_auc |> arrange(desc(auc))
```

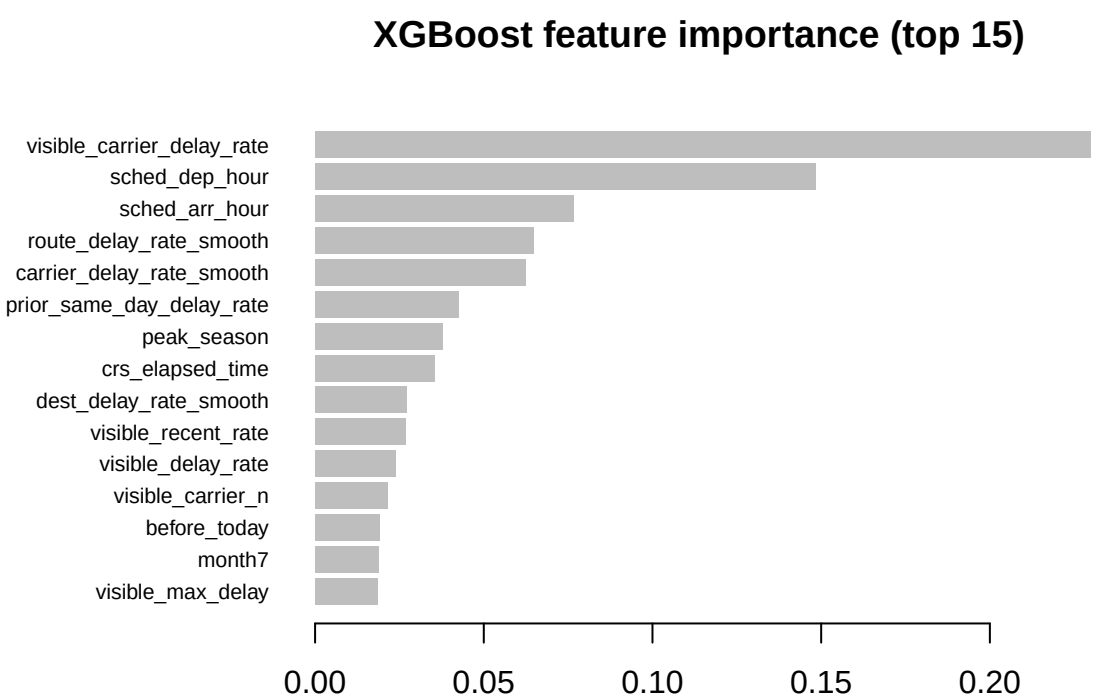
```
##            model_name       auc  accuracy
## 1 Ensemble (Weighted) 0.6947487 0.8861746
## 2      Ensemble (Avg) 0.6913612 0.8862799
## 3             Log Reg 0.6902226 0.8858587
## 4     XGBoost (tuned) 0.6896899 0.8861746
## 5               LASSO 0.6839234 0.8860693
## 6                  RF 0.6757789 0.8864905
## 7         Balanced RF 0.6075110 0.8862799
```

```r
vis_auc |> arrange(desc(accuracy))
```

```
##            model_name       auc  accuracy
## 1                  RF 0.6757789 0.8864905
## 2         Balanced RF 0.6075110 0.8862799
## 3      Ensemble (Avg) 0.6913612 0.8862799
## 4     XGBoost (tuned) 0.6896899 0.8861746
## 5 Ensemble (Weighted) 0.6947487 0.8861746
## 6               LASSO 0.6839234 0.8860693
## 7             Log Reg 0.6902226 0.8858587
```

XGBoost Feature Importance

```r
xgb.importance(model = xgb_model) |>
  xgb.plot.importance(top_n = 15, main = "XGBoost feature importance (top 15)")
```



```r
# Prepare guess data for model.matrix - dep_del15 is all NA bc it's what we're predicting
# model.matrix can't handle factors with < 2 levels, so we clean first
prep_guess <- function(df, ref_df) {
  # drop the target
  df <- df |> select(-any_of("dep_del15"))
  ref_df <- ref_df |> select(-any_of("dep_del15"))

  # for each factor column, force its levels to match the reference (train_split)
  # so model.matrix sees the same level set and doesn't throw error on single-level factors
  for (col in names(df)) {
    if (is.factor(ref_df[[col]]) && col %in% names(df)) {
      df[[col]] <- factor(as.character(df[[col]]),
                          levels = levels(ref_df[[col]]))
    }
  }
  df
}

guess_clean <- prep_guess(test_guess_model_df, train_split)
train_no_y <- train_split |> select(-dep_del15)

# XGBoost guess matrix - built from training columns as reference
guess_xgb_mats <- make_aligned_matrix(
  ~ .,
  train_no_y,
  guess_clean
)
dguess <- xgb.DMatrix(data = guess_xgb_mats$x_test)
xgb_guess <- predict(xgb_model, dguess)

# Logistic regression - predicts directly from data frame, no matrix needed
log_reg_guess <- predict(log_reg, type = "response", newdata = test_guess_model_df)

# LASSO guess matrix
guess_lm_mats <- make_aligned_matrix(
  ~ . + poly(sched_dep_hour, 2),
  train_no_y,
  guess_clean
)
lasso_guess <- predict(cv_lasso, newx = guess_lm_mats$x_test,
                       s = "lambda.min", type = "response") |> as.vector()

# RF / Balanced RF
test_guess_rf <- guess_clean |>
  mutate(dep_del15 = factor(NA, levels = c("0", "1")))
test_guess_rf <- align_to_training(test_guess_rf, train_split_rf)
rf_guess <- predict(rf_model, newdata = test_guess_rf, type = "prob")[, "1"]
brf_guess <- predict(brf_model, newdata = test_guess_rf, type = "prob")[, "1"]

# Weighted ensemble
delay.guesses <- 0.55 * xgb_guess +
                 0.20 * rf_guess +
                 0.15 * lasso_guess +
```

```r
                 0.10 * log_reg_guess

cat("Predictions:", length(delay.guesses),
    "| NAs:", sum(is.na(delay.guesses)), "\n")
```

```
## Predictions: 8687 | NAs: 0
```

```r
test_guess_fe |>
  summarise(
    pct_has_visible = mean(!is.na(visible_delay_rate)),
    mean_rate = mean(visible_delay_rate, na.rm = TRUE)
  )
```

```
## # A tibble: 1 x 2
##   pct_has_visible mean_rate
##             <dbl>     <dbl>
## 1               1     0.105
```

```r
# Save submission file
team.name <- "Amador"
test.acc <- 1 - mean(vis_auc$accuracy[vis_auc$model_name == "Ensemble (Weighted)"])

save(list = c("delay.guesses", "test.acc", "team.name"),
     file = paste0(team.name, "462final.RData"))
```